



Google Summer of Code '26 Project Proposal
Choudhury Mehbub Alam



Environmental NeTworked Sensor I: Customizable
dashboard, layouts, equations, export, registry.

Project Size: Large (350) hours

Mentors: [Colleen Josephson](#), [John Madden](#), [Alec Levy](#)

Contents

1. Understandings of ENTS-Backend Project	3
2. Overview of current codebase	3
3. Introduction.....	5
4. Problem Definition	5
5. Current State Background	6
6. Problems with Current Implementation	7
7. Proposed Solution	8
8. All of the proposed solutions in a nutshell.....	12
9. Implementation Plan.....	13
10. Project Methodology	15
11. Technical Elements	16
12. Challenges and Proposed Solutions	18
13. Project Goals.....	20
14. Objectives.....	20
15. Expected Deliverables	21
16. Future Work.....	23
17. PROJECT TIMELINE	23
18. Deliverables.....	24
19. Total Hours and Duration.....	24
20. Time Conflicts.....	25
21. Biographical Information	25
22. Relevant Experience and Educational Background	26
23. Technical Expertise	27
24. Project Engagement and Code Understanding	28
25. Contact Information	30

1. Understandings of ENTS-Backend Project

ENTS is not a generic dashboard app. It is a field-data platform where users need confidence in their data flow from collection to display. The backend receives data from multiple sensor families, stores data in PostgreSQL, and exposes it through Flask REST resources. The frontend is a React + Vite application that provides dashboard visualization, date range controls, and now partial unified chart rendering.

This project sits at that boundary. The target is not only visual polish. The target is to make user exploration (selecting cells, changing date ranges, comparing sensors, sharing views) reliable and fast, while also making chart code less repetitive and easier to evolve.

I also understand that ENTS is in a transitional architecture stage. The code already includes UnifiedChart and helper utilities for stream normalization, but the dashboard still combines legacy chart modules and repeated chart declarations. That means the right strategy is progressive modernization, not a risky all-at-once rewrite.

2. Overview of current codebase

The repository is organized into separate backend and frontend folders with clear boundaries:

- backend/api for Flask app, resources, schemas, models, and auth
- frontend/src for dashboard pages, chart components, and service calls
- tests for backend tests and frontend tests

Backend application setup in `backend/api/__init__.py` wires Flask, SQLAlchemy, Marshmallow, Flask-RESTful resources, OAuth/session setup, and SocketIO events. Route registration confirms active API surfaces for chart data retrieval:

- `/api/power/<int:cell_id>`
- `/api/teros/<int:cell_id>`
- `/api/sensor/`
- `/api/data-availability/`

On the frontend, dashboard composition currently lives in `dashboard/Dashboard.jsx`. This file still renders legacy chart components `PowerCharts` and `TerosCharts` while also rendering a long list of `UnifiedChart` entries with different type values (`power_voltage`, `power_current`, `teros12_vwc`, `teros12_ec`, `temperature`, `co2`, and others).

`UnifiedChart` currently contains a file-local `CHART_CONFIGS` object in `components/UnifiedChart.jsx`. It already captures chart metadata such as `sensor_name`, `measurements`, `units`, `axis IDs`, `chart ID`, and optional `axis policies`. This is a good foundation, and it strongly suggests that a registry-first architecture can be completed with limited risk.

Data services are currently split:

- `frontend/src/services/power.js`
- `frontend/src/services/teros.js`
- `frontend/src/services/sensor.js`

Data retrieval in chart components uses nested loops with awaited calls. This keeps code straightforward but serializes requests when many cells are selected.

3. Introduction

This proposal focuses on issue [#675](#) and closely related chart maintainability problems. The dashboard should become user-driven rather than developer-driven. Users should be able to arrange visual panels, save and share layouts, define equations for derived metrics, and export data with minimal friction.

At the same time, contributors should be able to add or modify chart types without touching many scattered files. This requires a stronger configuration boundary and more consistent data-fetch behavior.

I am proposing a scoped, realistic path that extends existing implementation patterns instead of replacing them. The work intentionally preserves core backend contracts while introducing new endpoints only as needed for the agreed deliverables (e.g. catalog first; optional layout/equation persistence if included in scope)

4. Problem Definition

The core problem can be summarized as a product-architecture mismatch.

The product now needs high user flexibility: custom dashboard arrangements, personal configuration, and shareability. However, the architecture still contains manual, component-level chart composition and repeated per-type render declarations. This

creates a gap between what the product must support and what the current code structure can evolve quickly.

In practical terms, this gap appears as:

- Slow iteration on new chart behaviors
- Repeated code paths for fetch-transform-render
- Harder long-term consistency across chart types
- Missing persistence model for layout and equation authoring

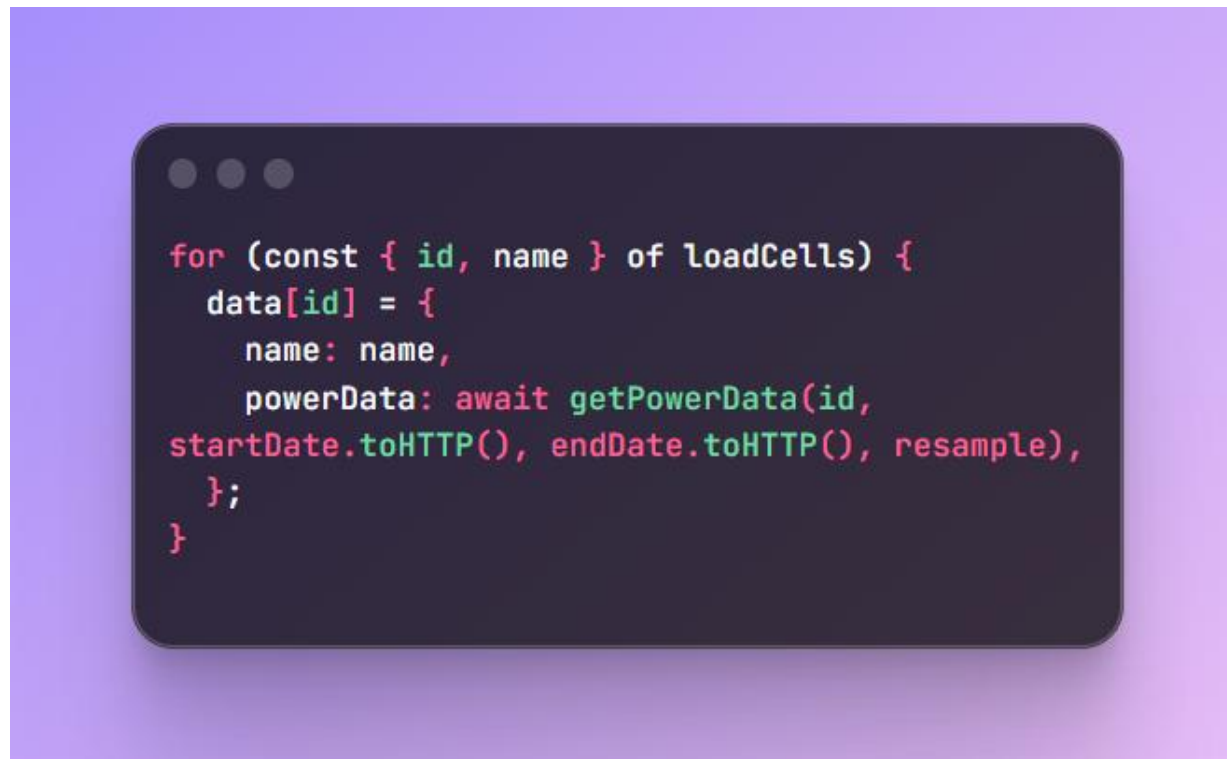
This proposal addresses both UX outcomes and the underlying technical constraints.

5. Current State Background

The current dashboard stack already includes several strengths

1. A working multi-chart dashboard with date range and stream support.
2. Existing utility helpers for stream normalization in `unifiedChartUtils.js`.
3. Stable backend resources that return power, teros, and generic sensor measurements with optional resampling.
4. Existing tests, including dashboard behavior tests (for example `DashboardSmartDate.test.jsx`).

Current fetch pattern example from existing code:



This pattern runs correctly, but for multiple cells it performs requests serially. In high-latency conditions, dashboard updates can feel slower than necessary.

6. Problems with Current Implementation

6.1 Repetition in chart composition

Dashboard.jsx currently includes many explicit UnifiedChart declarations. The list is understandable for now, but it does not scale cleanly when chart sets evolve or user-configurable layouts are introduced.

6.2 Configuration discoverability

CHART_CONFIGS inside UnifiedChart.jsx is useful but not shared as a standalone registry module. This increases coupling between rendering logic and metadata definitions.

6.3 Request performance under multi-cell selection

Sequential await loops in chart data retrieval increase total wait time under broader selections. This is a visible user experience cost when switching date windows or comparing multiple cells.

6.4 Missing persistence model for user customization

Issue [#675](#) requires layout save/share and equation authoring. These are not first-class persisted entities yet, so users cannot keep and reuse personalized analytical views.

6.5 Missing cohesive panel-level UX controls

Addressing this is intentionally lower priority than dynamic layout and database-driven chart discovery.

7. Proposed Solution

I propose a layered solution that keeps backward compatibility while enabling the needed user-facing capabilities.

7.1 Dynamic chart catalog (sensor-driven, not static frontend lists)

Instead of treating `CHART\_CONFIGS` in `UnifiedChart.jsx` as the long-term source of truth, the dashboard will build an effective “chart registry” from server-provided metadata. The backend already persists generic sensor channels in the `sensor` table (see `Sensor` model: `name`, `measurement`, `unit`, `cell\_id`). The project will add or extend a read-only API (for example, a dedicated endpoint

or an extension of existing cell/sensor discovery) that returns the distinct `(name, measurement, unit[, data\_type])` tuples available for the selected cell(s). The frontend will use that response to populate selectable series and to drive `UnifiedChart` (or a successor component) with minimal static duplication.

A small “presentation overlay” may remain in frontend code for edge cases (e.g. multi-axis layouts, display labels, axis policies such as VWC percent scaling) where the database does not encode layout intent—but the list of **which** sensors and measurements exist for a cell will not be hand-edited per new hardware type.

Note on transitional architecture: power and TEROS data may still be served from legacy tables in parallel with the generic `sensor`/`data` path. Until that data is fully unified, the catalog API can either merge discovery across sources or scope phase-one dynamic charts to generic `sensor` rows first; details will be agreed with mentors per component.

7.1b Priority order

- Highest: Dynamic charts and drag-and-drop layout (reorder/add/remove panels, persist layout—URL-first, optional saved layouts if agreed).
- High: Sensor-table-driven catalog and wiring charts to that discovery path; concurrent / batched fetching and stale-request guards for multi-cell views.

- Medium: Equations for derived series (with strict validation), CSV/data export for reporting.
- Lower: Pane-level styling (line color, markers, fine-grained axis cosmetics) as stretch work after core layout + dynamic series are stable.

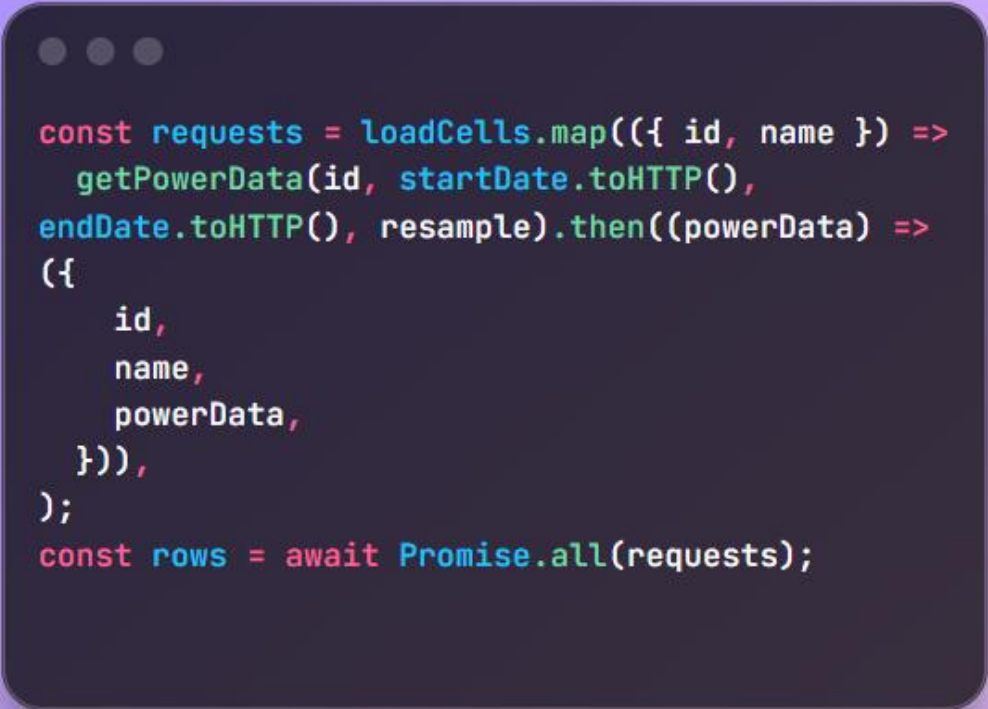
7.2 Concurrent fetch strategy

Replace sequential cell requests with concurrent Promise.all batching in chart components. The behavior remains the same, but response time improves significantly for multi-cell selections.

Target files:

- frontend/src/pages/dashboard/components/PowerCharts.jsx
- frontend/src/pages/dashboard/components/TerosCharts.jsx
- frontend/src/pages/dashboard/components/UnifiedChart.jsx

Proposed pattern:



```
const requests = loadCells.map(({ id, name }) =>
  getPowerData(id, startDate.toHTTP(),
    endDate.toHTTP(), resample).then((powerData) =>
    ({
      id,
      name,
      powerData,
    })),
);
const rows = await Promise.all(requests);
```

7.3 Layout management

Layout work is scheduled first in the implementation timeline: drag-and-drop (and optionally resize) of chart panels, with serialization to the URL for shareable views as the first persistence mechanism. Backend-persisted layouts (`/api/layouts` or equivalent) are a follow-on once the client-side layout model and URL round-trip are proven, unless mentors prefer server storage earlier for authenticated users.

7.4 Equation authoring and safety model

Add equation builder in the dashboard to support derived metrics and user-defined formula logic. Validation must be server-side with AST node allow-listing and bounded complexity.

Planned endpoint:

- /api/equations (POST, GET)

Equation design includes:

- expression text
- variable mapping to sensor streams
- optional metadata (name, chart scope, unit)
- ownership via authenticated user

7.5 Styling and export controls

Pane-level style controls (colors, line width, marker tweaks) are explicitly lower priority than dynamic layout and sensor-driven charts; they may be deferred to the second half of the project or reduced to minimal defaults. Export (e.g. CSV of loaded series) remains useful for field workflows and can stay medium priority behind layout + catalog integration.

7.6 Incremental rollout and compatibility

Legacy components remain available during migration to avoid sudden regressions. UnifiedChart and registry-based rendering will be introduced behind safe checkpoints.

8. All of the proposed solutions in a nutshell

In one sentence: convert ENTS dashboard from manually assembled chart rendering to a user-authorable, registry-driven, persistable dashboard experience.

In practical terms, that means:

- discovering plottable series from server metadata (principally the `sensor` table) instead of growing static frontend lists per new hardware type
- removing serial fetch bottlenecks where they hurt multi-cell use
- enabling drag-and-drop layout and shareable layout state (URL-first)
- enabling safe equations and practical data export
- treating pane-level styling as secondary to the above
- preserving existing backend data contracts while new endpoints are added only where agreed

This provides visible user value while reducing long-term contributor friction.

9. Implementation Plan

The implementation is intentionally staged so each phase can be reviewed and merged with low risk.

Review cadence: Every two weeks, mentors receive (a) a short written summary, (b) links to merged or open PRs, and (c) a demo or screen recording of the milestone. Odd weeks focus on implementation; even weeks end with “review & adjust” before starting the next milestone.

Week 1 — Milestone M0 (design sign-off)

- Finalize catalog API shape (fields, filtering by `cell\_id`, handling of legacy power/teros vs `sensor` table).
- Finalize layout state schema (panel id, chart binding, order) and URL encoding approach.
- Deliverable for review: design note + open questions list.

Week 2 — Milestone M1 (catalog consumption, read-only)

- Backend: list available `(name, measurement, unit)` for selected cells from `sensor` (and agreed legacy sources if required).
- Frontend: consume catalog to render a minimal dynamic list or picker (parity with static list not required yet).
- Deliverable for review: PR(s) + demo of API + UI proof.

Week 3-4 — Milestone M2 (drag-and-drop layout, URL persistence) [PRIMARY]

- Implement DnD reordering (and add/remove panels if in scope) using catalog-bound panels.
- Serialize/deserialize layout in URL; tests for round-trip.
- Deliverable for review: demo of shareable link restoring layout.

Week 5-6 — Milestone M3 (fetch performance & stability)

- Parallelize or batch per-cell fetches; cancellation on cell/date change; align with any upstream batch sensor API work.
- Deliverable for review: before/after notes + regression checks.

Week 7–8 — Milestone M4 (equations + validation) [if time; else start after M3]

- Equation UI + server-side allow-listed validation; scoped to derived series from already-loaded channels.
- Deliverable for review: PR + security notes.

Week 9–10 — Milestone M5 (export; optional saved layouts)

- CSV export of visible data; optional backend layout CRUD if not done earlier.
- Deliverable for review: user-facing short guide.

Week 11–12 — Hardening, documentation, stretch (pane styling only if M2–M5 are stable)

- Expanded tests, bug fixes, contributor note on how to add presentation-only overrides without static sensor lists.
- Final review package for mentors.

10. Project Methodology

My methodology is milestone-driven and verification-first.

10.1 Small batch delivery

I will avoid giant one-time PRs. Instead, each subsystem change will be a focused increment with clear before/after behavior and explicit rollback safety.

10.2 Contract-preserving changes

For existing endpoints that already serve dashboard data, the project will preserve request/response contracts unless an agreed extension is required.

10.3 Test-first risk control

Every structural refactor (registry extraction, render mapping, fetch changes) will include tests or behavior assertions before proceeding to the next phase.

10.4 Mentor sync cadence

Formal milestone reviews occur every two weeks per the implementation plan (written summary, PR links, demo or recording). In addition, I will post brief weekly check-ins (async) with what shipped and any blockers so mentors are never surprised between review points.

11. Technical Elements

11.1 Frontend elements

- React dashboard composition logic in Dashboard.jsx
- UnifiedChart configuration and stream utility integration
- Service layer modernization for concurrent request execution
- Layout and equation UI components
- State persistence for URL and user-saved layout objects

11.2 Backend elements

- Read-only catalog API (or extension of existing discovery) returning distinct (name, measurement, unit[, data_type]) per cell, with agreed handling for legacy power/teros if needed.
- Optional, if agreed after client-side layout and catalog milestones: Flask-RESTful resources for saved layouts and equations; Marshmallow (or equivalent) schema validation; SQLAlchemy models; Alembic migrations for new tables; auth and ownership checks for user-scoped records.

11.3 Data and model considerations

Layout entity will store:

- layout name
- chart list and positioning
- panel options (style, selected sensors, optional equation refs)
- owner identity

Equation entity will store:

- equation name
- expression
- variable bindings
- optional unit/metadata
- owner identity

11.4 Performance and reliability techniques

- Promise.all batching for multi-cell fetch
- stale update prevention and guarded setState flows
- reduced re-render pressure through scoped updates
- test cases for high-frequency dashboard interaction flows

12. Challenges and Proposed Solutions

Challenge 1: refactor risk in a live, mixed architecture

Proposed solution:

- maintain dual support during migration (legacy plus unified)
- switch to config-mapped rendering gradually
- verify parity before removing redundant paths

Challenge 2: equation safety and abuse resistance

Proposed solution:

- strict AST allow-list validation
- expression complexity limits
- server-side validation on save and evaluation

Challenge 3: state complexity with layout + style + equation coupling

Proposed solution:

- define versioned, minimal layout schema
- validate schema before persistence
- isolate panel state transformations through utility functions

Challenge 4: ensuring dashboard remains responsive

Proposed solution:

- parallel fetch strategy
- render-path optimization
- benchmark before/after key user flows

Challenge 5: maintaining contributor friendliness

Proposed solution:

- clear registry structure and documentation
- examples for adding chart types without touching many files
- concise architecture notes for future maintainers

Challenge 6: Stream/unified ingest uses display types (e.g. POWER_VOLTAGE) while the `sensor` table stores ingest `name` and

`measurement` strings that may not match legacy `CHART\_CONFIGS` keys one-to-one.

Proposed solution: define an explicit mapping layer in one place (backend catalog serializer or frontend adapter), documented and tested; reduce special cases over time as data model unification proceeds.

13. Project Goals

The project has five top-level goals:

- (1) Drag-and-drop dynamic dashboard layout and shareable layout state.
- (2) Chart population from database-backed sensor discovery (`sensor` table and agreed supplements).
- (3) Performance and reliability of multi-cell loading.
- (4) Equations and export.
- (5) Pane-level styling only as time allows.

14. Objectives

Objective A: user ownership

- users can rearrange, save, and share dashboards

Objective B: analytical flexibility

- users can define and apply equations for derived metrics safely

Objective C: export and optional presentation

- users can export chart data (e.g. CSV) without external tools; pane-level styling remains stretch-only after core layout and catalog work.

Objective D: codebase maintainability

- chart behavior is driven by reusable registry and cleaner composition patterns

Objective E: performance improvement

- reduce waiting time for multi-cell refresh paths through concurrent fetching

15. Expected Deliverables

Frontend deliverables (ordered by priority):

- Drag-and-drop dashboard layout with URL-persisted layout state (and optional server-saved layouts if agreed).
- Dynamic chart panel binding driven by server-provided sensor/measurement catalog (sourced from the `sensor` table, with a defined story for legacy power/teros during transition).
- Concurrent or batched fetching and stale-update handling for multi-cell selections.
- Equation authoring UI with validated derived series (scope agreed with mentors).

- CSV export of loaded chart data.
- Pane-level style controls only as stretch after core milestones.

Backend deliverables:

- Read API for sensor/measurement catalog per cell (and migration path noted if split across `sensor` vs legacy tables).
- Optional layout and equation persistence, schemas, migrations, tests—only as agreed after client prototype.

Documentation deliverables:

- contributor guide for chart registry usage
- user guide for layouts, equations, and export
- implementation notes for migration behavior and compatibility

Quality deliverables:

- benchmark comparison before/after fetch changes
- expanded frontend dashboard behavior tests
- endpoint and validation test coverage for new resources

16. Future Work

After this project, ENTS can evolve further with lower effort because the dashboard foundation will be more modular.

Potential extensions:

- role-based default layouts by team/research group
- reusable equation libraries
- alerting rules based on equation thresholds
- richer annotation layers on charts
- advanced downsampling strategies for very long windows
- dataset export presets for recurring analyses

These features are intentionally not included in this proposal scope to keep delivery realistic and high quality.

17. PROJECT TIMELINE

The timeline is 12 weeks of active development and review, designed around GSoC pacing and frequent checkpoints. Each week includes coding, test updates, and mentor sync.

I prefer milestone completion over broad parallel work so that each delivered section is stable before the next subsystem is opened.

18. Deliverables

By the end of the project, deliverables are expected to align with the priority order in 7.1b: (1) drag-and-drop dashboard layout with URL-persisted layout state; (2) dynamic chart panels driven by a server-provided catalog based on the `sensor` table (with an agreed approach for legacy power/teros during transition); (3) improved multi-cell fetch behavior (parallel/batch) and stale-update handling; (4) equation authoring with server-side validation, scope as agreed with mentors; (5) CSV export of loaded series. Backend layout/equation persistence and pane-level styling are optional or stretch deliverables only if core milestones are met first.

19. Total Hours and Duration

Planned duration: 12 weeks

Planned effort: 350 hours total

Effort model:

- Weeks 1-10: about 30 hours each week (300 hours)
- Weeks 11-12: about 25 hours each week (50 hours)

Distribution:

- 60 percent implementation,
- 25 percent testing and debugging,
- 15 percent documentation and mentor communication.

20. Time Conflicts

I currently have no known major conflicts that would significantly reduce my committed project time. My university semester exams are expected around June (exact dates yet to be announced). I will plan ahead to ensure minimal disruption during that period.

If any unavoidable conflict arises, I will communicate it promptly and adjust my weekly goals to stay aligned with project milestones.

I treat schedule transparency as an essential part of engineering reliability and will proactively keep mentors informed.

21. Biographical Information

I, Choudhury Mehbub Alam, am a final-year undergraduate student pursuing a B.Tech in Computer Science and Business Systems at the Academy of Technology, Kolkata (2022–2026). I am a full-stack developer with a strong focus on building scalable, production-grade applications.

I currently work as a Software Development Engineer at House of EdTech, where I contribute to the development of a mutual fund investment platform. My work spans frontend engineering, backend API development, and cloud deployment, with a strong emphasis on performance, scalability, and user experience.

I am also an open-source contributor and have contributed to research-oriented platforms such as ENTS (UC Santa Cruz JLab Sensing Lab), improving usability and data visualization features

22. Relevant Experience and Educational Background

Education:

- B.Tech in Computer Science and Business Systems
Academy of Technology, Kolkata (2022 – 2026)

Professional Experience:

- **Software Development Engineer – House of EdTech (Oct 2025 – Present)**
 - Leading feature development for a production-grade mutual fund platform
 - Optimizing performance and scalability across frontend and backend
 - Mentoring interns and contributing to system design decisions
- **Full-Stack Developer Intern – House of EdTech (Jul 2025 – Sep 2025)**
 - Built a complete mutual fund platform using Next.js (TypeScript), Tailwind CSS, and ShadCN UI
 - Integrated eKYC and e-sign workflows, improving onboarding conversion by ~30%
 - Developed secure APIs using Supabase and Next.js
 - Deployed infrastructure on AWS (ECS Fargate, S3) with Dockerized pipelines

- Built a Telegram bot for automated notifications with high reliability (99% uptime)

Open Source & Research Contributions:

- Contributor to ENTS Backend (UC Santa Cruz JLab Sensing Lab)
 - Improved UX (drag-and-drop), added chart export features, and fixed UI/visibility issues
- Project Maintainer – Apertre 2.0
 - Reviewed 60+ PRs and managed contributor collaboration across repositories

23. Technical Expertise

- **Languages:** TypeScript, JavaScript, Python, C++
- **Frontend Development:** React, Next.js, Tailwind CSS, HTML, CSS
- **Backend Development:** Node.js, Express.js, Flask
- **Databases & ORM:** PostgreSQL, MongoDB, Prisma, Mongoose, Supabase
- **Cloud & DevOps:** AWS (ECS Fargate, S3), Docker, CI/CD pipelines
- **Tools & Platforms:** Git, Postman, Vercel, Render

Core Strengths:

- Building scalable full-stack applications
- Designing secure APIs and data access layers (RLS, auth flows)
- Cloud-native deployment and containerization
- Performance optimization and real-time systems

- Developer-focused product building (SaaS platforms, analytics dashboards)

24. Project Engagement and Code Understanding

My understanding of ENTS is not only from reading files. It is also based on direct contribution history in this repository under the devleo10 identity. I reviewed repository history and branch traces to connect this proposal to real implementation work and previously merged pull requests.

Verified merged PRs:

- PR [#404](#) (toggle-feature-in-dashboard)
- PR [#410](#) (feature/graph-export)
- PR [#413](#) (improved-archive-overlay)
- PR [#427](#) (fix-archive-section)

Issues:

- Issue [#722](#) (open): Refactor conftest: cell setup in init_database and teardown cleanup
- Issue [#721](#) (open): Secure cell creation: require auth on POST /cell/ and set owner from authenticated user
- Issue [#425](#) (closed): Fix Archive Modal Scrolling and Z-Index
- Issue [#423](#) (closed): Add Unit Tests for Import Cell Data Function

- Issue [#400](#) (closed): SQLAlchemy Warning: Overlapping Many-to-Many Relationship in Cell and Tag Models

PR still left to merge (open):

- PR [#732](#) (open): Catches decode/processing failures in sensor ingest

These PRs show practical engagement in the exact area this proposal targets: dashboard behavior, user-facing interaction quality, and panel-level usability improvements. They also show familiarity with ENTS frontend conventions and the tradeoffs involved in modifying live dashboard flows without destabilizing the rest of the application.

In addition to contribution history, I validated current implementation constraints directly from code:

- mixed legacy and unified chart rendering in Dashboard composition
- file-local chart config structure in UnifiedChart
- sequential fetch patterns in chart update routines
- active backend endpoint wiring for `/api/power`, `/api/teros`, `/api/sensor`, and `/api/data-availability`

This is why the proposal emphasizes incremental integration instead of a risky full rewrite. The plan is intentionally structured around repository realities: keep working paths stable, introduce user-facing capabilities in controlled phases, and attach test and benchmark evidence at each milestone.

I will continue using the same evidence-driven style throughout the project by tying each design claim to implementation artifacts, test updates, and measurable behavior changes.

25. Contact Information

Location: Kolkata, West Bengal, India

Email: mehbubwork@gmail.com

Phone: +91 86178 13516

GitHub: <https://github.com/devleo10>

LinkedIn: <https://www.linkedin.com/in/choudhury-mehbub-alam-b6b191219/>

Portfolio: <https://takealookat.me>